

Topic: Classes in Java

A class is a group of objects that has common properties. It is a template or blueprint from which objects are created.

A class in java can contain:

- variables
- methods
- constructors
- blocks
- classes and interfaces

Syntax to declare a class:

```
access-modifier class <class_name>{  
    //variables  
    //methods  
}
```

Example:

```
public class Person {  
    // variables  
    String name;  
  
    // methods  
    void printName() {  
        System.out.println(name);  
    }  
}
```

Topic: Objects in Java

- An entity that has state and behavior is known as an object. It can be physical or logical (tangible and intangible).
 - e.g. chair, bike, marker, pen, table, car etc.
 - The example of intangible object is banking system.
- Object is an instance of a class.
- Class is a template or blueprint from which objects are created. So object is the instance (result) of a class.

An object has three characteristics:

- **state:** represents data (value) of an object.
- **behavior:** represents the behavior (functionality) of an object such as deposit, withdraw etc.
- **identity:** Object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. But, it is used internally by the JVM to identify each object uniquely.

Example: Pen is an object. Its name is Reynolds, color is white etc. known as its state. It is used to write, so writing is its behavior.

Topic: Defining Classes and Objects

a) Object and Class Example: main within the class:

```
//Defining a Student class.
class Student{
//defining variables
    int id;//field or data member or instance variable
    String name;
//creating main method inside the Student class
    public static void main(String args[]){
//Creating an object or instance
        Student s1=new Student();
// accessing members through reference variable
        System.out.println(s1.id);
        System.out.println(s1.name);
    }
}
```

Output:

0

Null

- ✓ In the above example, we have created a ***Student class*** which has two data members ***id*** and ***name***.
- ✓ We are creating the object of the Student class by ***new*** keyword and printing the object's value.
- ✓ Here, we are creating a ***main()*** method inside the class.

b) Object and Class Example: main outside the class

- ✓ In real time development, we create classes and use it from another class.
- ✓ It is a better approach than previous one. Let us see a simple example, where we are having ***main()*** method in another class.

```
//Defining Student class
class Student{
    int id;
    String name;
}
//Creating another class TestStudent which contains the main method
class TestStudent{
    public static void main(String args[]){
        Student s1=new Student();
        System.out.println(s1.id);
        System.out.println(s1.name);
    }
}
```

Output:

0

Null

Topic: Different ways of Creating Objects in JAVA

Different ways to create objects in java:

1. Using new keyword:

This is the most common way to create an object in java. Almost 99% of objects are created in this way.

Eg: `MyObject object=new Object();`

2. Using Class.forName():

If we know the name of the class & if it has a public default constructor we can create an object in this way.

Syntax:

```
MyObject obj=(MyObject) class.forName("object").newInstance();
```

3. Using clone():

The clone() can be used to create a copy of an existing object.

Syntax:

```
MyObject obj=new MyObject();
```

```
MyObject object=(MyObject )obj.clone();
```

4. Using Object Deserialization:

Object deserialization is nothing but creating an object from its serialized form.

Syntax:

```
objectInputStream istream=new objectInputStream(some data);
```

```
MyObject object=(MyObject) instream.readObject();
```

5. Using newInstance() method

```
Object obj =
```

```
DemoClass.class.getClassLoader().loadClass("DemoClass").newInstance();
```

Topic: Assigning one object to another (Object Cloning)

Object cloning refers to the creation of an exact copy of an object. It creates a new instance of the class of the current object and initializes all its fields with exactly the contents of the corresponding fields of this object.

Methods for Performing Object Cloning in Java

There are 2 methods for creating Object Cloning in Java that are mentioned below:

1. Using Assignment Operator to create a copy of the reference variable
2. Creating a copy using the clone() method

1. Using Assignment Operator to create a copy of the reference variable

In Java, there is no operator to create a copy of an object. If we use the assignment operator then it will create a copy of the reference variable and not the object.

// Java program to demonstrate copying an object using assignment operator

```
class Test {
    int x, y;
    Test() {
        x = 10;
        y = 20;
    }
}

// Driver Class
class Main {
    public static void main(String[] args) {
        Test ob1 = new Test();
        System.out.println(ob1.x + " " + ob1.y);
        // Creating a new reference variable ob2
        // pointing to same address as ob1
        Test ob2 = ob1;
        // Any change made in ob2 will be reflected in ob1
        ob2.x = 100;
        System.out.println(ob1.x + " " + ob1.y);
        System.out.println(ob2.x + " " + ob2.y);
    }
}
```

Output

```
10 20
100 20
100 20
```

2. Creating a copy using the clone() method

The class whose object's copy is to be made must have a public clone method in it or in one of its parent classes.

- Every class that implements clone() should call super.clone() to obtain the cloned object reference.
- The class must also implement java.lang.Cloneable interface whose object clone we want to create otherwise it will throw CloneNotSupportedException when the clone method is called on that class's object.

Usage of clone() method -Shallow Copy

In the below code example the clone() method does create a completely new object with a different hashCode value, which means its in a separate memory location.

```
// A Java program to demonstrate usage of clone() method
class Test {
    int x, y;
}

// Contains a reference of Test and implements clone.
class Test2 implements Cloneable {
    int a;
    int b;
    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

// Driver class
public class Main {
    public static void main(String args[])
        throws CloneNotSupportedException {
        Test2 t1 = new Test2();
        t1.a = 10;
        t1.b = 20;

        // Creating a copy of object t1 and passing it to t2
        Test2 t2 = (Test2)t1.clone();
        t2.a = 100;
        // Change in primitive type of t2 will not be
        reflected in t1 field
        System.out.println(t1.a + " " + t1.b);
        System.out.println(t2.a + " " + t2.b);
    }
}
```

Output:

10 20

100 20

Topic: Access Control (or) Access Modifiers in Java

What are Access Modifiers?

In Java, access modifiers are used to set the accessibility (visibility) of classes, interfaces, variables, methods, constructors, data members, and the setter methods.

Access modifiers are mainly used for encapsulation. It can help us to control what part of a program can access the members of a class. So that misuse of data can be prevented.

For example,

```
class Animal {  
    public void method1() {...}  
  
    private void method2() {...}  
}
```

In the above example, we have declared 2 methods: `method1()` and `method2()`. Here,

- *method1* is `public` - This means it can be accessed by other classes.
- *method2* is `private` - This means it can not be accessed by other classes.

Types of Access Modifier

Modifier	Description
default	declarations are visible only within the package (package private)
private	declarations are visible within the class only
protected	declarations are visible within the package or all subclasses
public	declarations are visible everywhere

Summarized view of Accessibility of Access modifier in Java

Accessibility	default	private	protected	public
Inside Same Class	Yes	Yes	Yes	Yes
Subclass of a Class	Yes	No	Yes	Yes
Same Package Different Class	Yes	No	Yes	Yes
Different Package's Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

1. default Access Modifier

If we do not explicitly specify any access modifier for classes, methods, variables, etc, then by default the default access modifier is considered. For example,

```
package defaultPackage;
class Logger {
    void message(){
        System.out.println("This is a message");
    }
}
```

Here, the *Logger* class has the default access modifier. And the class is visible to all the classes that belong to the *defaultPackage* package. However, if we try to use the *Logger* class in another class outside of *defaultPackage*, we will get a compilation error.

2. private Access Modifier

When variables and methods are declared *private*, they cannot be accessed outside of the class. For example,

```
class Data {
    // private variable
    private String name;
}

public class Main {
    public static void main(String[] main) {

        // create an object of Data
        Data d = new Data();

        // access private variable and field from another class
        d.name = "LBRCE";
    }
}
```

In the above example, we have declared a private variable named *name*. When we run the program, we will get the following error:

```
Main.java:18: error: name has private access in Data
    d.name = "LBRCE";
    ^
```

The error is generated because we are trying to access the private variable of the *Data* class from the *Main* class.

If we need to access those private variables, we can use the getters and setters method. For example,

```
class Data {
    private String name;
```

```
// getter method
public String getName() {
    return this.name;
}
// setter method
public void setName(String name) {
    this.name= name;
}
}
public class Main {
    public static void main(String[] main){
        Data d = new Data();

        // access the private variable using the getter and setter
        d.setName("LBRCE");
        System.out.println(d.getName());
    }
}
```

Output:

LBRCE

In the above example, we have a private variable named *name*. In order to access the variable from the outer class, we have used methods: `getName()` and `setName()`. These methods are called getter and setter in Java.

3. protected Access Modifier

When methods and data members are declared `protected`, we can access them within the same package as well as from subclasses. For example,

```
class Animal {
    // protected method
    protected void display() {
        System.out.println("I am an animal");
    }
}

class Dog extends Animal {
    public static void main(String[] args) {

        // create an object of Dog class
        Dog dog = new Dog();
        // access protected method
        dog.display();
    }
}
```

Output:

I am an animal

In the above example, we have a protected method named `display()` inside the *Animal* class. The *Animal* class is inherited by the *Dog* class

We then created an object *dog* of the *Dog* class. Using the object we tried to access the protected method of the parent class.

Since protected methods can be accessed from the child classes, we are able to access the method of *Animal* class from the *Dog* class.

Note: We cannot declare classes or interfaces `protected` in Java.

4. public Access Modifier

When methods, variables, classes, and so on are declared `public`, then we can access them from anywhere. The public access modifier has no scope restriction. For example,

```
// Animal.java file
// public class
public class Animal {
    // public variable
    public int legCount;

    // public method
    public void display() {
        System.out.println("I am an animal.");
        System.out.println("I have " + legCount + " legs.");
    }
}

// Main.java
public class Main {
    public static void main( String[] args ) {
        // accessing the public class
        Animal animal = new Animal();

        // accessing the public variable
        animal.legCount = 4;
        // accessing the public method
        animal.display();
    }
}
```

Output:

```
I am an animal.
I have 4 legs.
```

Here,

- The public class *Animal* is accessed from the *Main* class.
- The public variable *legCount* is accessed from the *Main* class.
- The public method `display()` is accessed from the *Main* class.

Topic: Accessing Private Members of Class

a) Using setter and getter methods:

Since private members cannot be accessed directly from outside the class, you need to use public methods (often called getters and setters) to interact with them:

- **Getters:** Methods that return the value of a private field.
- **Setters:** Methods that modify the value of a private field, often with validation logic

Example:

```
public class Main {
    public static void main(String[] args) {
        // Create an instance of Person
        Person person = new Person("Alice", 30);

        // Access private fields through public methods
        System.out.println("Name: " + person.getName());
        System.out.println("Age: " + person.getAge());

        // Modify private fields through public methods
        person.setName("Bob");
        person.setAge(25);

        // Display updated information
        person.displayInfo(); // Outputs: Name: Bob, Age: 25
    }
}
```

b) Using java.reflection API

You can access the private methods of a class using java reflection package.

Step1 – Instantiate the **Method** class of the **java.lang.reflect** package by passing the method name of the method which is declared private.

Step2 – Set the method accessible by passing value **true** to the **setAccessible()** method.

Step3 – Finally, invoke the method using the **invoke()** method.

Example

```
import java.lang.reflect.Method;

public class DemoTest {
    private void sampleMethod() {
        System.out.println("hello");
    }
}

public class SampleTest {
```

```
public static void main(String args[]) throws Exception {  
    Class c = Class.forName("DemoTest");  
    Object obj = c.newInstance();  
    Method method = c.getDeclaredMethod("sampleMethod",  
null);  
    method.setAccessible(true);  
    method.invoke(obj, null);  
}
```